Estácio

2º Procedimento | Interface Cadastral com Servlet e JSPs

Nome da disciplina: Desenv. Back-end Corporativo Com Java e Cloud

Campus: POLO ELDORADO III - CONTAGEM – MG

Curso: DESENVOLVIMENTO FULL STACK

Turma: DGT2811 – 9001

2026/1

Guilherme Rogério Souza Comunian

Link do Repositório

18/04/2026

Sumário

Objetivo Geral 3

1 - Criar um Servlet com o nome ServletProdutoFC, no projeto CadastroEE-war 3

2 - Criar a página de consulta, com o nome ProdutoLista.jsp 5

3 - Criar a página de cadastro, com o nome ProdutoDados.jsp 6

4 - Testar as funcionalidades do sistema: 7

5 - Análise e Conclusão 8

a. Como funciona o padrão Front Controller, e como ele é implementado em um aplicativo Web Java, na arquitetura MVC? 8

b. Quais as diferenças e semelhanças entre Servlets e JSPs? 8

c. Qual a diferença entre um redirecionamento simples e o uso do método forward, a partir do RequestDispatcher? Para que servem parâmetros e atributos nos objetos HttpRequest?...9

Objetivo Geral

O objetivo geral deste trabalho é compreender e aplicar os conceitos da arquitetura MVC (Model-View-Controller) em uma aplicação web Java, utilizando Servlets e JSPs, com foco na implementação do padrão Front Controller.

Busca-se desenvolver uma aplicação organizada em camadas, onde o controle das requisições HTTP é centralizado em um único controlador, permitindo a separação clara entre lógica de negócio, controle de fluxo e apresentação.

Além disso, o trabalho tem como objetivo aprofundar o entendimento sobre o funcionamento de requisições web, incluindo o uso de parâmetros e atributos no HttpServletRequest, bem como a diferença entre os mecanismos de redirecionamento (redirect) e encaminhamento interno (forward), contribuindo para a construção de sistemas web mais estruturados, escaláveis e de fácil manutenção.

1 - Criar um Servlet com o nome ServletProdutoFC, no projeto CadastroEE-war

```
/// Imports:
// ... Line 15
import cadastroee.model.Produto;
import jakarta.ejb.EJB;
import cadatroee.controller.ProdutoFacadeLocal;
import jakarta.servlet.RequestDispatcher;
// ... Line 24
```

```java
@WebServlet(name = "ServletProdutoFC", urlPatterns = {"/ServletProdutoFC"})
public class ServletProdutoFC extends HttpServlet {
    @EJB
    private ProdutoFacadeLocal facade;

    protected void processRequest(HttpServletRequest request, HttpServletResponse
response) {
        String action = request.getParameter("action");

        String destiny;
        switch (action){
            case "formAlterar":
            case "formIncluir":
                destiny = "ProdutoDados.jsp";
            break;
            default:
                destiny = "ProdutoLista.jsp";
            break;
        }

        Integer id = null;
        Produto product = new Produto();
        switch (action) {

            case "listar":
                request.setAttribute("list", facade.findAll());
            break;
            case "formAlterar":
                id = Integer.parseInt(request.getParameter("id"));
                request.setAttribute("product", facade.find(id));
            break;
            case "excluir":
                id = Integer.parseInt(request.getParameter("id"));
                facade.remove(facade.find(id));
                request.setAttribute("list", facade.findAll());
            break;
            case "alterar":
                id = Integer.parseInt(request.getParameter("id"));
                product = facade.find(id);

                product.setName(request.getParameter("name"));

 product.setPreco(Float.parseFloat(request.getParameter("price").replace(",", ".")));

                facade.edit(product);

                request.setAttribute("list", facade.findAll());
            break;
            case "incluir":
                product.setName(request.getParameter("name"));

 product.setPreco(Float.parseFloat(request.getParameter("price").replace(",", ".")));

                facade.create(product);
                request.setAttribute("list", facade.findAll());
            break;
        }
        try {
            RequestDispatcher rd = request.getRequestDispatcher(destiny);
            rd.forward(request, response);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    // ... Servlet Methods
}
```

2 - Criar a página de consulta, com o nome ProdutoLista.jsp

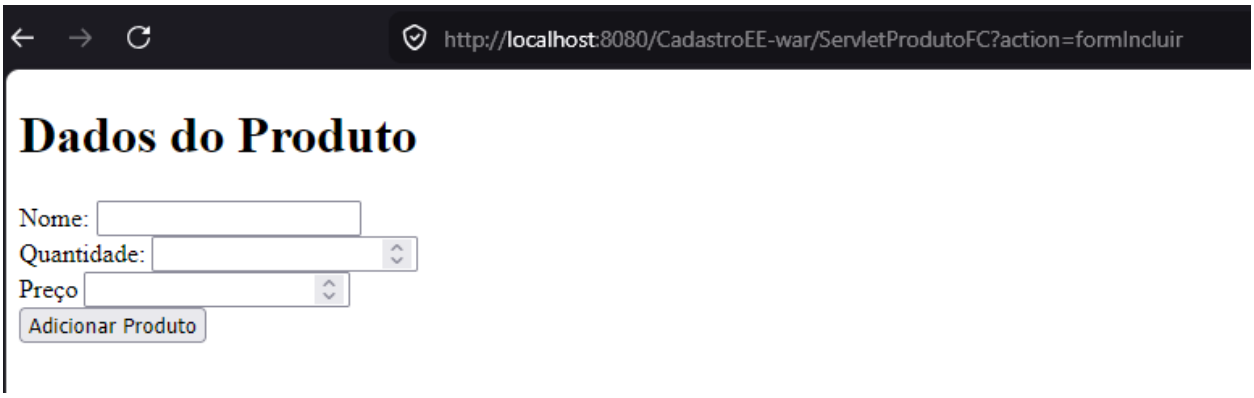
```jsp
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Products</title>
    </head>
    <body>
        <h1>Listagem Produtos</h1>
        <a href="ServletProdutoFC?action=formIncluir">Novo Produto</a>
        <%@ page import="java.util.List" %>
        <%@ page import="cadastroee.model.Produto" %>

        <%
            List<Produto> lista = (List<Produto>) request.getAttribute("list");
        %>

        <table>
            <tr>
                <th>ID</th>
                <th>Nome</th>
                <th>Quantidade</th>
                <th>Preço</th>
                <th>Ações</th>
            </tr>
            <%
                if (lista != null) {
                    for (Produto p : lista) {
            %>
            <tr>
                <td><%= p.getId() %></td>
                <td><%= p.getName() %></td>
                <td><%= p.getQtd() %></td>
                <td><%= p.getPreco() %></td>
                <td>
                    <a href="ServletProdutoFC?action=formAlterar&id=<%= p.getId() %>">
                        Alterar
                    </a>

                    |

                    <a href="ServletProdutoFC?action=excluir&id=<%= p.getId() %>">
                        Excluir
                    </a>
                </td>
            </tr>

            <%
                    }
                }
            %>
        </table>
    </body>
</html>
```

3 - Criar a página de cadastro, com o nome ProdutoDados.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ page import="cadastroee.model.Produto" %>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Dados Produto</title>
        <%
            Produto product = (Produto) request.getAttribute("product");
            String action = (product == null) ? "incluir" : "alterar";
        %>
    </head>
    <body>
        <h1>Dados do Produto</h1>
        <form method="post" action="ServletProdutoFC">
            <div>
                <label for="name">Nome:</label>
                <input type="text" name="name" id="name" <%= (product != null)? String.format("value=\"%s\"", product.getName()) :
"" %>>
            </div>
            <div>
                <label for="qtd">Quantidade:</label>
                <input type="number" name="qtd" id="qtd" min="0" <%= (product != null)? String.format("value=\"%d\"",
product.getQtd()) : "" %>>
            </div>
            <div>
                <label for="price">Preço</label>
                <input type="number" name="price" id="price" min="0.0" <%= (product != null)? String.format("value=\"%.2f\"",
product.getPreco()) : "" %>>
            </div>
            <div hidden>
                <%if("alterar".equals(action)) {%>
                    <input name="id" id="id" value="<%= product.getId() %>">
                <%}%>
                <input name="action" id="action" value="<%= action %>">
            </div>
            <button type="submit"><%= (product == null)? "Adicionar Produto" : "Editar Produto" %></button>
        </form>
    </body>
</html>
```

Alteração:

Insert:



4 - Testar as funcionalidades do sistema:

Durante a execução do sistema web Java desenvolvido, foram realizados testes nas principais funcionalidades da aplicação, incluindo o fluxo de cadastro, listagem e alteração de dados.

Os resultados observados foram:

- O Servlet Front Controller recebeu todas as requisições da aplicação corretamente.
- As ações foram identificadas através do parâmetro action na URL.
- O sistema conseguiu redirecionar ou encaminhar corretamente para as páginas JSP correspondentes.
- Os dados enviados via formulários foram processados e armazenados corretamente (quando aplicável ao banco de dados).
- Em casos de erro de entrada (como parâmetros vazios ou inválidos), ocorreram exceções como NumberFormatException, indicando a necessidade de validação dos dados antes do processamento.

Exemplo observado:

- Requisição com ID inválido ou vazio gerou erro interno no servidor: java.lang.NumberFormatException: For input string: ""

Isso reforça a importância da validação de parâmetros no Controller antes da conversão de tipos.

5 - Análise e Conclusão

a. Como funciona o padrão Front Controller, e como ele é implementado em um aplicativo Web Java, na arquitetura MVC?

O padrão Front Controller consiste em um único ponto de entrada para todas as requisições da aplicação web. Em vez de múltiplos servlets para cada funcionalidade, todas as requisições passam por um único controlador central.

Funcionamento no MVC:

- O Front Controller (Servlet principal) recebe todas as requisições HTTP.
- Ele interpreta a ação solicitada (geralmente via parâmetro action).
- Encaminha a requisição para o componente adequado (Service, DAO ou JSP).
- Retorna a resposta ao usuário.

Arquitetura MVC:

- Model: representa os dados e regras de negócio.
- View (JSP): responsável pela interface com o usuário.
- Controller (Servlet Front Controller): gerencia o fluxo entre Model e View.

b. Quais as diferenças e semelhanças entre Servlets e JSPs?

Semelhanças:

- Ambos fazem parte da tecnologia Java Web.
- Ambos podem gerar conteúdo dinâmico.
- Ambos executam no servidor (container como GlassFish ou Tomcat).
- Podem acessar objetos como HttpServletRequest e HttpServletResponse.

Diferenças:

Servlets	JSPs
Escrito em Java puro	Mistura HTML com Java (scriptlet ou tags)
Focado em lógica de controle	Focado em apresentação (View)
Mais difícil para HTML	Mais simples para interface
Melhor para processamento	Melhor para exibição

c. Qual a diferença entre um redirecionamento simples e o uso do método forward, a partir do RequestDispatcher? Para que servem parâmetros e atributos nos objetos HttpRequest?

Forward (encaminhamento interno):

- Ocorre dentro do servidor.
- A URL não muda no navegador.
- A requisição continua a mesma.
- Mais rápido, pois não cria nova requisição.

Redirect (redirecionamento):

- O servidor manda o navegador fazer uma nova requisição.
- A URL muda no navegador.
- Perde os atributos da requisição original.

Parâmetros:

- Vêm da requisição HTTP (URL ou formulário).
- São imutáveis (somente leitura).
- Sempre são Strings.

Atributos:

- Criados dentro do servidor (backend).
- Podem armazenar qualquer objeto.
- Usados para compartilhar dados entre Servlet e JSP.